

Revisão para a Prova

Desenvolvimento de Sistemas Web II com Go

Prova: 11/06 (qui) · Multiprova · 25 questões fechadas · 40 min

Consulta: documentação oficial do Go + 1 folha A4 manuscrita

Como cai a prova

- **Escopo:** Unidades 1 e 2
- **Entra:** HTTP, Go web, net/http, Chi, middleware, JSON/validação, sqlc, Repository, relacionamentos 1:N
- **NÃO entra:** JWT/autenticação, testes automatizados, segurança/OWASP (*cobrados no projeto e na Lista 5+6*)
- **Pesos:** Conceitos ~50% · Persistência ~30% · Leitura de código ~20%
- 40 min para 25 questões $\approx$ **1,5 min por questão** — não trave em uma só

1. HTTP: métodos e idempotência

- **Idempotentes:** GET , PUT , DELETE — repetir não muda o resultado final
- **Não idempotentes:** POST (cria a cada chamada), PATCH (em geral)
- **Seguros** (não alteram estado): GET , HEAD
- Pegadinha clássica: "*POST é idempotente*" → **falso**

2. Status codes que você precisa saber

Código	Quando usar
200 OK	Sucesso com corpo
201 Created	Recurso criado
204 No Content	Sucesso sem corpo
400 Bad Request	Entrada inválida
404 Not Found	Recurso inexistente
422 Unprocessable	Validação semântica falhou
500 Internal	Erro do servidor

3. http.Handler e HandlerFunc

```
type Handler interface {  
    ServeHTTP(w http.ResponseWriter, r *http.Request)  
}
```

- `http.HandlerFunc` é um **tipo função** que implementa `Handler`
- Permite usar uma função comum como handler:

```
http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) { ... })
```

4. Roteamento net/http (Go 1.22+)

- Padrão = **método + caminho** com wildcards `{}` :

```
mux.HandleFunc("GET /users/{id}", h) // só GET
```

- Lê o segmento da rota com `r.PathValue` :

```
id := r.PathValue("id")
```

- Erros comuns: usar `:id` (sintaxe de outro framework) ou esquecer o método.

5. Query params vs Path vs Body

```
// /search?status=active
status := r.URL.Query().Get("status") // query

id := r.PathValue("id")                // segmento da rota

json.NewDecoder(r.Body).Decode(&dto)   // corpo JSON
```

- Não confunda `Query().Get` (query string) com `PathValue` (rota).

6. Middleware: a assinatura idiomática

```
func mw(next http.Handler) http.Handler {  
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
        // antes  
        next.ServeHTTP(w, r)  
        // depois  
    })  
}
```

- Tipo: **func(http.Handler) http.Handler** — vale para net/http e Chi.

7. Ordem de execução do middleware

- Encadeamento `Logger → Auth → handler` :
 - **Logger** executa primeiro, depois **Auth**, depois o **handler**
- Pense em **camadas de cebola**: entra de fora pra dentro, sai de dentro pra fora
- O `next.ServeHTTP(w, r)` é o ponto que "passa adiante"

8. context.Context da requisição

- `r.Context()` propaga **cancelamento, deadlines e valores** escopados à requisição
- Usos típicos: passar dados do middleware ao handler, cancelar query no banco
- **Não** serve para definir status code nem substituir o corpo

```
ctx := context.WithValue(r.Context(), key, val)
r = r.WithContext(ctx)
```

9. JSON: struct tags

```
type User struct {  
    ID      int    `json:"id"`  
    Name    string `json:"name"`  
    Email   string `json:"- "`      // omitido na serialização  
}
```

- `json:"- "` → **não aparece** no JSON
- `json:",omitempty"` → omite se valor zero
- Sem tag → campo **exportado** sai com o nome do campo

10. Codificar e decodificar JSON

```
// Ler corpo da requisição  
json.NewDecoder(r.Body).Decode(&dto)  
  
// Escrever resposta  
json.NewEncoder(w).Encode(payload)
```

- Em handlers, prefira **Decoder/Encoder** (stream) a **Marshal / Unmarshal**.
- Defina **Content-Type** e **WriteHeader** **antes** de escrever o corpo.

11. Chi: parâmetros e organização de rotas

```
chi.URLParam(r, "orderId")    // parâmetro de rota no Chi

r.Route("/users", func(r chi.Router) { ... }) // subrouter + prefixo
r.Group(func(r chi.Router) { ... })           // middleware sem prefixo
r.Mount("/admin", adminRouter)                // anexa um handler/subrouter
```

- **Route:** novo prefixo · **Group:** agrupa middleware · **Mount:** monta subrouter

12. Validação com go-playground/validator

```
type CreateUser struct {  
    Name  string `validate:"required"`  
    Email string `validate:"required,email"`  
    Age   int    `validate:"gte=18"`  
}
```

- Tags separadas por **vírgula**: `validate:"required,email"`
- É a tag **validate** (não `json`, não `binding`)

13. Erros de API: RFC 7807

- Corpo padronizado de erro → `Content-Type: application/problem+json`
- Campos comuns: `type`, `title`, `status`, `detail`, `instance`
- Vantagem: clientes tratam erros de forma uniforme

14. sqlc: o que é (e o que não é)

- **É:** gerador de **código Go type-safe** a partir de SQL que **você** escreve
- **Não é:** ORM, nem driver de conexão, nem framework web
- Anotações nas queries:

```
-- name: GetUser :one      -- retorna no máx. 1 linha
-- name: ListUsers :many   -- retorna várias
-- name: DeleteUser :exec  -- não retorna linhas
```

15. Padrão Repository

- **Objetivo:** abstrair o acesso a dados atrás de uma **interface**
- Desacopla a **camada de serviço** da **persistência**
- Facilita troca de implementação e organização do código

```
type UserRepo interface {  
    GetByID(ctx context.Context, id int) (User, error)  
    List(ctx context.Context) ([]User, error)  
}
```

16. Modelagem 1:N

- Relação `author (1) → book (N)` :
 - A **chave estrangeira** vai no lado N: `book.author_id`
- Erros comuns: pôr a FK no lado "1", ou criar tabela de junção (isso é N:N)

17. JOINS: LEFT vs INNER

- **INNER JOIN**: só linhas com correspondência nos dois lados
- **LEFT JOIN**: todas as linhas da tabela à esquerda, mesmo **sem** correspondência
- "Liste **todos** os autores, inclusive sem livros" → **LEFT JOIN**

```
SELECT a.*, b.*  
FROM author a  
LEFT JOIN book b ON b.author_id = a.id;
```

18. Montar 1:N em Go (cuidado com N+1)

- JOIN devolve **linhas repetidas** (uma por filho)
- Agrupe por id do pai em um `map[int]*Author`, anexando cada `Book`
- **Anti-padrão N+1**: uma query por filho dentro de um loop → evite

19. Paginação por deslocamento

```
SELECT * FROM books  
ORDER BY id  
LIMIT $1 OFFSET $2;
```

- `LIMIT` = tamanho da página · `OFFSET` = quantos pular
- Sintaxe do PostgreSQL — nada de `TOP` / `SKIP`

20. Armadilhas frequentes

- Confundir **idempotência** dos métodos HTTP
- Trocar `PathValue` por `Query().Get`
- Inverter a **ordem** dos middlewares
- Esquecer o **método** no padrão de rota do Go 1.22
- Pôr a **FK no lado errado** do 1:N
- Achar que `INNER JOIN` traz pais sem filhos

21. Antes da prova

- Monte sua **folha A4** com: tabela de status codes, assinatura de middleware, anotações sqlc, esqueleto de 1:N + JOIN
- Lembre que pode consultar a **doc oficial do Go**
- Releia as **Listas 1–4**
- Gerencie o tempo: ~1,5 min/questão; marque e volte nas difíceis

Boa prova! 🚀